Algoritmos e Lógica de Programação

Aula 04 — Vetores (Arrays), List e Stack

2º Semestre de 2025

Prof. Marcelo Amorim

FUND. 1902

FECAP

Recapitulando: laços de repetição

- while repete enquanto a condição for verdadeira; for reúne inicialização, condição e atualização.
- Laços podem ser aninhados, e break / continue interrompem ou pulam uma repetição.
- Nos exercícios, cada valor lido sobrescrevia o anterior: só guardávamos o valor atual.

Hoje vamos guardar vários valores de uma vez, em uma única variável, e percorrê-los com os laços que já conhecemos.

Por que usar arrays?

Para ler 5 notas e mostrar todas no final, precisaríamos de uma variável para cada uma:

```
int nota1 = Convert.ToInt32(Console.ReadLine());
int nota2 = Convert.ToInt32(Console.ReadLine());
int nota3 = Convert.ToInt32(Console.ReadLine());
int nota4 = Convert.ToInt32(Console.ReadLine());
int nota5 = Convert.ToInt32(Console.ReadLine());
// e para 100 notas? E se o número de notas mudar?
```

Um array guarda vários valores do mesmo tipo em uma única variável, e podemos percorrê-lo com um laço.

O que é um array (vetor)?

Um array é um conjunto de posições numeradas, cada uma guardando um valor do mesmo tipo.

`int[] notas`



- Todos os elementos têm o mesmo tipo (int, double, string...).
- O tamanho é definido na criação e não muda depois.
- As posições (índices) começam em 0: o último índice é Length - 1.

Declaração, criação e inicialização

Criando vazio

```
int[] notas = new int[5];

// 5 posições, todas com o valor padrão
```

Criando com valores

```
int[] notas = { 7, 8, 9 };

// 3 posições, já com valores
```

Valores padrão de um array criado com new:

int	double	bool	string
0	0.0	false	null

Acessando e alterando elementos

```
int[] notas = { 7, 8, 9 };
Console.WriteLine(notas[0]);       // 7
notas[2] = 10;                     // altera a 3ª posição
Console.WriteLine(notas[2]);       // 10
Console.WriteLine(notas.Length);   // 3
```

- notas[i] lê ou altera o elemento que está na posição i.
- notas.Length informa quantos elementos o array tem.
- Como o primeiro índice é 0, a "3ª posição" é notas[2].

Percorrendo um array com for

Preenchendo (lendo do usuário)

```
int[] notas = new int[5];

for (int i = 0; i < notas.Length; i++)
{
    Console.Write($"Nota {i + 1}: ");
    notas[i] = Convert.ToInt32(Console.ReadLine());
}
```

Exibindo

```
for (int i = 0; i < notas.Length; i++)
{
    Console.WriteLine($"notas[{i}] = {notas[i]}");
}
```

A condição usa i < notas.Length (e não <=), porque o último índice válido é Length - 1.

Cuidado com o IndexOutOfRangeException

Acessar uma posição que não existe gera um erro em tempo de execução:

```
int[] notas = new int[5];   // índices válidos: 0, 1, 2, 3, 4
notas[5] = 10;              // erro! não existe a posição 5
// System.IndexOutOfRangeException
```

Erro clássico no laço

```
for (int i = 0; i <= notas.Length; i++)   // erro comum: <= vai até o índice 5
```

Regra: os índices válidos vão de 0 até Length - 1.

foreach: percorrendo sem índice

Com foreach

```
int[] notas = { 7, 8, 9 };
foreach (int nota in notas)
{
    Console.WriteLine(nota);
}
```

Equivalente com for

```
for (int i = 0; i < notas.Length; i++)
{
    Console.WriteLine(notas[i]);
}
```

- O foreach visita todos os elementos, do primeiro ao último, sem precisar de índice.
- A variável do foreach (nota) é somente leitura: não dá para atribuir um novo valor a ela.
- Use foreach só para ler; use for quando precisar do índice ou alterar elementos.

string[]: arrays de texto

```
string[] nomes = { "Ana", "Bruno", "Carla" };

Console.WriteLine(nomes[1]);         // Bruno
Console.WriteLine(nomes[1].Length);  // 5

foreach (string nome in nomes)
{
    Console.WriteLine(nome.ToUpper());
}
```

- Cada elemento é uma string, então todos os métodos de string (ToUpper, Trim, Length...) funcionam em nomes[i].
- Um string[] criado com new começa com elementos null (e não com texto vazio).

Demo 1: menu e tabela com vetores

Menu no console

```
string[] opcoes = { "Somar", "Subtrair", "Multiplicar", "Sair" };

Console.WriteLine("--- MENU ---");
for (int i = 0; i < opcoes.Length; i++)
{
    Console.WriteLine($"{i + 1}) {opcoes[i]}");
}
```

Saída

```
--- MENU ---
1) Somar
2) Subtrair
3) Multiplicar
4) Sair
```

Tabela (opcional, com Spectre.Console)

```
var tabela = new Table();
tabela.AddColumn("Nº");
tabela.AddColumn("Opção");
for (int i = 0; i < opcoes.Length; i++)
{
    tabela.AddRow((i + 1).ToString(), opcoes[i]);
}
AnsiConsole.Write(tabela);
```

Saída (aparência aproximada)

```
+----+-------------+
| Nº | Opção       |
+----+-------------+
| 1  | Somar       |
| 2  | Subtrair    |
| 3  | Multiplicar |
| 4  | Sair        |
+----+-------------+
```

Demo 2: produtos, preços e sacola

```
string[] produtos = { "Arroz", "Feijão", "Leite", "Café" };
double[] precos   = { 25.90, 8.50, 5.20, 18.00 };
int[] sacola = { 0, 2, 2, 3 };  // Arroz, Leite, Leite, Café

double total = 0;
for (int i = 0; i < sacola.Length; i++)
{
    int indice = sacola[i];
    Console.WriteLine($"{produtos[indice]} - R$ {precos[indice]:F2}");
    total += precos[indice];
}
Console.WriteLine($"Total: R$ {total:F2}");
```

Saída

```
Arroz - R$ 25,90
Leite - R$ 5,20
Leite - R$ 5,20
Café - R$ 18,00
Total: R$ 54,30
```

- produtos e precos são paralelos: mesmo índice = mesmo produto.
- sacola guarda apenas índices: sacola[i] aponta para uma posição de produtos e de precos.
- O índice 2 aparece duas vezes: o Leite entra duas vezes no total.

O limite do tamanho fixo

E se não soubermos quantos itens vão entrar, como os produtos que o usuário coloca na sacola?

```
int[] sacola = new int[3];
sacola[0] = 2;
sacola[1] = 0;
sacola[2] = 3;
sacola[3] = 1;   // erro: a sacola só comporta 3 itens (IndexOutOfRangeException)
```

Alternativas:

- Criar um array enorme "por garantia" desperdiça memória e ainda pode não bastar.
- Usar uma coleção de tamanho dinâmico, como List<T>, que cresce e diminui conforme a necessidade.

List<T>: lista de tamanho dinâmico

```
List<string> sacola = new List<string>();

sacola.Add("Arroz");
sacola.Add("Leite");
sacola.Remove("Arroz");

Console.WriteLine(sacola.Count);             // 1
Console.WriteLine(sacola.Contains("Leite")); // True
Console.WriteLine(sacola[0]);                // Leite
```

Principais membros

- Add(x): adiciona no final.
- Remove(x): remove a primeira ocorrência.
- Contains(x): informa se existe.
- Count: quantidade de itens (o "Length" da lista).
- lista[i]: acesso por índice, como no array.

Se o compilador não encontrar List<T>, adicione using System.Collections.Generic; no topo do arquivo.

Demo 3: sacola de compras com List

```
List<string> sacola = new List<string>();

while (true)
{
    Console.Write("Produto (ou 'sair'): ");
    string produto = Console.ReadLine();
    if (produto == "sair") break;
    sacola.Add(produto);
}

Console.WriteLine("Sua sacola:");
foreach (string item in sacola)
{
    Console.WriteLine($"- {item}");
}
Console.WriteLine($"Total de itens: {sacola.Count}");
```

Saída

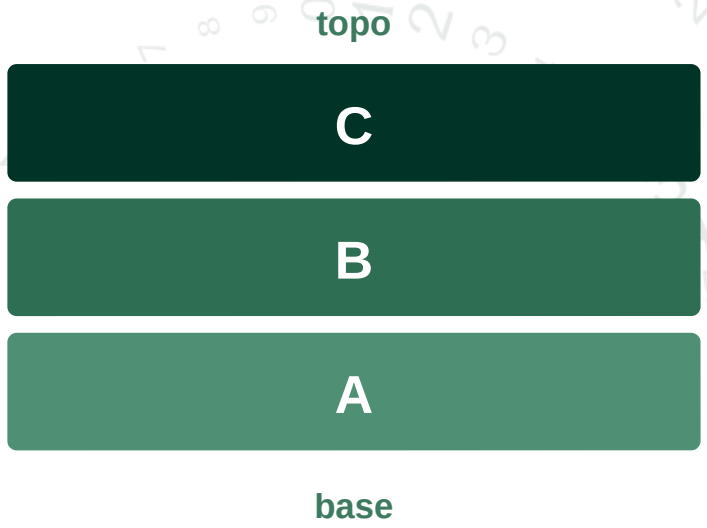
```
Produto (ou 'sair'): Arroz
Produto (ou 'sair'): Leite
Produto (ou 'sair'): sair
Sua sacola:
- Arroz
- Leite
Total de itens: 2
```

- while (true) + break (da Aula 03): repete até digitar sair.
- Add guarda cada produto; não precisamos saber quantos serão.
- foreach percorre a sacola para listar.

Stack<T>: a pilha

- Stack = pilha: o último a entrar é o primeiro a sair (LIFO), como uma pilha de pratos.
- Push empilha; Pop remove o topo e devolve o valor; Peek só olha o topo.
- Pop ou Peek em pilha vazia causam erro: confira Count > 0 antes.

```
Stack<string> pilha = new Stack<string>();
pilha.Push("A");
pilha.Push("B");
pilha.Push("C");
Console.WriteLine(pilha.Peek());  // C (só olha o topo)
Console.WriteLine(pilha.Pop());   // C (remove o topo)
Console.WriteLine(pilha.Count);   // 2
```



Array x List x Stack

Array

- Tamanho fixo, definido na criação.
- Acesso por índice: v[i].
- Use quando o tamanho é conhecido.

List<T>

- Tamanho dinâmico: Add e Remove.
- Acesso por índice: lista[i].
- Use quando a quantidade de itens varia.

Stack<T>

- Tamanho dinâmico: Push e Pop.
- Acessa só o topo (LIFO).
- Use quando importa o último a entrar: desfazer, inverter.

Exercício 1

Ler e exibir em ordem inversa

Ler 5 números inteiros, guardá-los em um array e exibi-los em ordem inversa.

Exemplo

```
Entrada: 3, 8, 1, 9, 4
Saída: 4, 9, 1, 8, 3
```

Dica: percorra o array do último índice (Length - 1) até 0.

Exercício 2

Soma, média e acima da média

Ler 5 números inteiros em um array e exibir a soma, a média e quantos valores estão acima da média.

Exemplo

```
Entrada: 4, 8, 6, 2, 10
Saída: Soma = 30 | Média = 6,0 | Acima da média: 2
```

Dica: use um laço para somar e outro, depois, para contar os valores maiores que a média.

Exercício 3

Maior e menor com posição

Ler 5 números inteiros em um array e exibir o maior e o menor valor, com a posição (índice) de cada um.

Exemplo

```
Entrada: 4, 9, -2, 7, 5
Saída: Maior = 9 (posição 1) | Menor = -2 (posição 2)
```

Dica: comece supondo que o maior e o menor valor estão na posição 0.

Exercício 4

Buscar um valor

Ler 5 números inteiros em um array e depois um valor a buscar. Informar a posição em que foi encontrado, ou "não encontrado".

Exemplo

```
Array: 3, 8, 1, 9, 4
Buscar 9  ->  Encontrado na posição 3
Buscar 7  ->  Não encontrado
```

Dica: percorra o array e use break assim que encontrar o valor.

Exercício 5

Lista de nomes (string[])

Crie um array com 5 nomes (definidos no código). Leia um nome e informe se ele está na lista, ignorando maiúsculas, minúsculas e espaços extras.

Exemplo

```
nomes = Ana, Bruno, Carla, Diego, Elisa
Entrada: "  carla "  ->  Saída: Carla está na lista (posição 2)
Entrada: "Maria"  ->  Saída: Maria não está na lista
```

Ferramentas / lembrete:

- .Trim() e .ToUpper() (ou .ToLower()), aplicados nos dois lados da comparação.
- O == já compara o valor de strings em C#.

Exercício 6

Lista de tarefas (List<string>)

Crie uma lista de tarefas com List<string>. Exiba um menu com 1) Adicionar, 2) Remover, 3) Listar e 4) Sair, repetindo até o usuário escolher Sair.

Exemplo

```
Escolha: 1  ->  Tarefa: Estudar C#
Escolha: 1  ->  Tarefa: Fazer exercícios
Escolha: 3  ->  1 - Estudar C#  |  2 - Fazer exercícios
Escolha: 2  ->  Remover: Estudar C#
Escolha: 4  ->  Encerrando...
```

Ferramentas / lembrete:

- Add, Remove e Count do List<string>.
- Combine while + switch, como no menu de operações da aula anterior.

Exercícios de fixação (extra)

Para quem quiser praticar mais em casa.

Palíndromo com Stack

Ler uma palavra, empilhar cada caractere em um Stack<char>, desempilhar formando a palavra invertida e comparar com a original. Exemplo: "arara" é palíndromo; "casa" não é.

Array em ordem crescente

Ler 5 números em um array e informar se estão em ordem crescente (cada elemento maior ou igual ao anterior). Exemplo: 1, 3, 3, 8, 9 -> sim; 1, 5, 2, 8, 9 -> não.

Recapitulando

Hoje vimos:

- Arrays: tamanho fixo, elementos do mesmo tipo e índices a partir de 0.
- Como percorrer com for e foreach, e o cuidado com o IndexOutOfRangeException.
- string[] e arrays paralelos (produtos, preços e sacola com índices).
- List<T> para tamanho dinâmico e Stack<T> para a lógica de pilha (LIFO).
- Prática com 6 exercícios e os demos de menu, tabela e sacola de compras.

Próxima aula: Funções